

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL XI
EXCEPTION HANDLING



Disusun Oleh:

Gionaldo Candrawansah 105224031

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Praktikum Modul 11 membahas tentang *Error Handling* atau Penanganan Eksepsi dalam bahasa Java. Eksepsi (*Exception*) adalah kejadian tidak terduga yang mengganggu alur normal eksekusi program. Jika tidak ditangani, program akan mengalami *crash* atau berhenti mendadak. Pada Take-Home Task ini, mahasiswa ditugaskan merancang aplikasi *Command Line Interface* (CLI) reservasi tiket kereta bernama "JAVA EXPRESS" yang tangguh dan kebal terhadap berbagai *error* operasional maupun *input* pengguna. Implementasinya mencakup pembuatan *Custom Exception* (baik *Checked* maupun *Unchecked*), penggunaan kata kunci *throw* dan *throws*, serta struktur kendali *try-catch-finally* untuk memastikan program dapat memberikan pesan *error* yang informatif, mencegah *infinite loop*, dan menjamin pembersihan memori (*resource leak prevention*).

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	kode, nama, rute	String	Menyimpan identitas kelas Kereta seperti kode rute, nama kereta, dan relasinya.
2	sisaKursi	int	Menyimpan kapasitas kursi yang tersedia pada suatu objek Kereta.
3	daftarKereta	List<Kereta>	Menyimpan daftar objek kereta api yang aktif menggunakan struktur data <i>Collections</i> .
4	namaKereta, sisaKursi (Exception)	String, int	Menyimpan <i>state</i> tambahan di dalam kustom eksepsi <i>TiketHabisException</i> untuk informasi <i>error</i> detail.
5	scanner	Scanner	Objek untuk membaca <i>input</i> pengguna dari terminal.
6	isRunning	boolean	Variabel <i>flag</i> untuk mengontrol

			perulangan <i>while</i> menu utama (<i>infinite loop</i> terkendali).
7	pilihan	int	Menyimpan angka <i>input</i> menu yang dipilih oleh pengguna.
8	nik, namaPenumpang	String	Variabel lokal untuk menyimpan data pengguna saat mengisi form pemesanan.
9	jumlahTiket	int	Variabel lokal untuk jumlah tiket yang ingin dipesan pengguna.

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	DataPenumpangTidakValidException(), RuteTidakDitemukanException(), TiketHabisException())	Constructor	Menginisialisasi pesan <i>error</i> kustom (dan data tambahan) saat dieksekusi dengan memanggil <i>super(message)</i> .
2	Kereta()	Constructor	Menginisialisasi <i>blueprint</i> kereta beserta kapasitas awalnya.
3	SistemReservasi()	Constructor	Membuat instansiasi <i>ArrayList</i> dan mengisi daftar rute kereta <i>default</i> .
4	kurangiKursi()	Procedural	Mengurangi nilai atribut <i>sisaKursi</i> setelah pemesanan dikonfirmasi sukses.
5	tampilkanInfo(),	Procedural	Mencetak format

	lihatJadwal()		tabel rapi dari jadwal dan ketersediaan kursi kereta api ke layar.
6	pesanTiket()	Procedural (dengan throws)	Menjalankan logika bisnis pemesanan, melempar (throw) eksepsi bila validasi gagal, serta mencetak <i>E-Ticket</i> bila berhasil.
7	main()	Procedural	Titik mula berjalannya simulasi I/O interaktif aplikasi JAVA EXPRESS yang dibungkus dengan <i>Error Handling</i> .

IV. Dokumentasi dan Pembahasan Code

Berikut adalah kode sumber yang digunakan untuk Sistem Reservasi JAVA EXPRESS:

```
import java.util.*;
```

```
// a. Unchecked Exception (Mewarisi RuntimeException)
```

```
class DataPenumpangTidakValidException extends RuntimeException {
    public DataPenumpangTidakValidException(String message) {
        super(message);
    }
}
```

```
// b. Checked Exception (Mewarisi Exception)
```

```
class RuteTidakDitemukanException extends Exception {
    public RuteTidakDitemukanException(String message) {
        super(message);
    }
}
```

```
// c. Checked Exception Khusus (Mewarisi Exception dan Menyimpan State/Info)
```

```
class TiketHabisException extends Exception {
    private String namaKereta;
    private int sisaKursi;

    public TiketHabisException(String message, String namaKereta, int sisaKursi) {
        super(message);
    }
}
```

```

        this.namaKereta = namaKereta;
        this.sisaKursi = sisaKursi;
    }

    public String getNamaKereta() { return namaKereta; }
    public int getSisaKursi() { return sisaKursi; }
}

// 2. KELAS KERETA API
class Kereta {
    private String kode;
    private String nama;
    private String rute;
    private int sisaKursi;

    public Kereta(String kode, String nama, String rute, int kapasitas) {
        this.kode = kode;
        this.nama = nama;
        this.rute = rute;
        this.sisaKursi = kapasitas;
    }

    public String getKode() { return kode; }
    public String getNama() { return nama; }
    public String getRute() { return rute; }
    public int getSisaKursi() { return sisaKursi; }

    public void kurangiKursi(int jumlah) {
        this.sisaKursi -= jumlah;
    }

    public void tampilkanInfo() {
        System.out.printf("| %-5s | %-15s | %-12s | %-10d |\n", kode, nama, rute, sisaKursi);
    }
}

// 3. KELAS SISTEM RESERVASI
class SistemReservasi {
    private List<Kereta> daftarKereta;

    public SistemReservasi() {
        daftarKereta = new ArrayList<>();
        daftarKereta.add(new Kereta("K01", "Argo Bromo", "JKT - SBY", 50));
        daftarKereta.add(new Kereta("K02", "Parahyangan", "JKT - BDG", 15));
    }

    public void lihatJadwal() {
        System.out.println("          JADWAL KERETA API AKTIF          ");
    }
}

```

```

        System.out.printf("| %-5s | %-15s | %-12s | %-10s |\n", "KODE", "NAMA KERETA", "RUTE",
"SISA KURSI");
        System.out.println("-----");
        for (Kereta k : daftarKereta) {
            k.tampilkanInfo();
        }
    }

    public void pesanTiket(String kode, String nik, String namaPenumpang, int jumlahTiket)
        throws RuteTidakDitemukanException, TiketHabisException {

        System.out.println("\n[SISTEM] Memproses reservasi Anda...");

        // 1. Validasi NIK (Unchecked Exception ditaruh lebih awal)
        if (nik.length() != 16 || !nik.matches("\\d+")) {
            throw new DataPenumpangTidakValidException("Penolakan Sistem: NIK harus terdiri dari
TEPAT 16 digit angka tanpa huruf/symbol!");
        }

        // 2. Cari Kereta
        Kereta keretaPilihan = null;
        for (Kereta k : daftarKereta) {
            if (k.getKode().equalsIgnoreCase(kode)) {
                keretaPilihan = k;
                break;
            }
        }

        // Validasi Kereta (Checked Exception)
        if (keretaPilihan == null) {
            throw new RuteTidakDitemukanException("Penolakan Sistem: Rute dengan kode " + kode +
"" tidak terdaftar di jadwal kami.");
        }

        // 3. Validasi Kapasitas Kursi (Checked Exception)
        if (jumlahTiket > keretaPilihan.getSisaKursi()) {
            throw new TiketHabisException("Penolakan Sistem: Jumlah pemesanan melebihi kapasitas
yang tersedia.",
                keretaPilihan.getNama(),
                keretaPilihan.getSisaKursi());
        }

        // 4. Proses Berhasil
        keretaPilihan.kurangiKursi(jumlahTiket);
        String idTransaksi = "TRX-" + UUID.randomUUID().toString().substring(0, 5).toUpperCase();

        System.out.println("      E-TICKET JAVA EXPRESS      ");
        System.out.println("ID Transaksi : " + idTransaksi);

```



```

        System.out.print("Masukkan Jumlah Tiket : ");

        int jumlah = scanner.nextInt();
        scanner.nextLine();

        sistem.pesanTiket(kode, nik, nama, jumlah);

    }
    catch (InputMismatchException e) {
        System.out.println("\n[ERROR INPUT] Jumlah tiket harus berupa ANGKA
BULAT!");
        scanner.nextLine();
    }
    catch (DataPenumpangTidakValidException e) {
        System.out.println("\n[VALIDASI GAGAL] " + e.getMessage());
    }
    catch (RuteTidakDitemukanException e) {
        System.out.println("\n[RUTE GAGAL] " + e.getMessage());
    }
    catch (TiketHabisException e) {
        System.out.println("\n[STOK GAGAL] " + e.getMessage());
        System.out.println("-> Info : Kereta " + e.getNamaKereta() + " saat ini hanya
tersisa " + e.getSisaKursi() + " kursi.");
    }
    catch (Exception e) {
        System.out.println("\n[SYSTEM ERROR] Terjadi kesalahan fatal: " +
e.getMessage());
    }
    break;
case 0:
    isRunning = false;
    break;
default:
    System.out.println("\n[PERINGATAN] Menu tidak valid. Silakan pilih 1, 2, atau 0.");
}
}
} finally {
    if (scanner != null) {
        scanner.close();
    }
    System.out.println(" [SYSTEM LOG] Blok penanganan akhir (finally) dieksekusi.");
    System.out.println(" Sesi JAVA EXPRESS telah ditutup. Memori telah dibersihkan.");
}
}
}

```

Pembahasan:

1. Penerapan *Custom Exception*: Sistem ini mengimplementasikan tiga buah *domain-specific exception*. `DataPenumpangTidakValidException` diwariskan dari `RuntimeException` menjadikannya tipe *Unchecked* (tidak dipaksa harus dibungkus try-catch saat *compile*). Sebaliknya, `RuteTidakDitemukanException` dan `TiketHabisException` diwariskan dari kelas induk `Exception` menjadikannya *Checked Exception*.
2. Deklarasi `throws` dan Penggunaan `throw`: Pada *method* `pesanTiket()`, kita menggunakan kata kunci `throws` di bagian *signature* untuk mendelegasikan *Checked Exception* kepada si pemanggil fungsi. Di dalam badan fungsinya, kita menggunakan `throw new` untuk membangkitkan objek *error* secara proaktif saat validasi NIK, kode rute, dan kuota sisa kursi gagal dipenuhi. Hal ini memisahkan murni *business logic* dari logika *error*.
3. Blok Multi *Catch*: Pada kelas `Main`, pembacaan masukan dilakukan di dalam blok `try`. Penggunaan blok `catch` bertingkat dibuat berurutan, mulai dari `InputMismatchException` (kesalahan *typing* pengguna dari `Scanner`), hingga *Exception* buatan kita, serta menaruh penangkap `Exception e` (kelas akar) di bagian paling bawah sebagai garda terakhir jika ada *error* yang tidak terprediksi.
4. Pembersihan *Buffer Scanner*: Dalam setiap tangkapan `InputMismatchException`, disematkan perintah `scanner.nextLine()` agar sisa *enter/newline* dari huruf yang *error* terbuang dari memori, hal ini adalah taktik untuk menghindari program terjebak *infinite loop*.
5. Blok *Finally*: Program utama dilindungi oleh sebuah blok `try` besar, di mana pasangannya adalah blok `finally`. Blok ini dijamin pasti tereksekusi baik program berjalan mulus hingga pengguna keluar via menu, maupun jika sistem hancur karena *fatal crash*. Di dalamnya, dilakukan `scanner.close()` untuk mengakhiri fungsi I/O dengan aman dan mencegah kebocoran memori (*memory leak*).

V. Kesimpulan

Melalui tugas implementasi *Error Handling* ini, dapat disimpulkan bahwa penanganan eksepsi adalah lapis krusial dalam rekayasa perangkat lunak. Tanpa try-catch, *input* remeh seperti karakter tak terduga dapat membuat sistem "Java Express" berhenti seketika. Pembuatan kelas *Custom Exception* membuktikan bahwa kita bisa menciptakan arsitektur kode yang lebih ekspresif dan deskriptif karena setiap kesalahan ditandai dengan identitas spesifik domain bisnis (contoh: `TiketHabisException`). Lebih jauh, penggunaan blok `finally` secara efisien mendemonstrasikan praktik pemrograman yang baik dalam menjaga integritas *resource* di dalam JVM.

VI. Daftar Pustaka

1. Timotheus Ferdinand Tjondrojo & Yan Stephen Christian Immanuel Hutagalung. (2026). *Modul II - Error Handling*. Laboratorium Komputer, Program Studi Ilmu Komputer, Universitas Pertamina.
2. Laboratorium Komputer Universitas Pertamina. (2026). *Take Home Task - Sistem Reservasi Tiket Kereta Api "JAVA EXPRESS"*. Program Studi Ilmu Komputer.